SE Lab 4: Vibe Coding

Name: Abhigyan Dutta
SRN: PES2UG24CS019
Class: 5A, CSE
Date: 16 October 2026
Assigned repo: SETAPESU26/17_wordle (scenario 17, Wordle)
Tool: Claude (Claude Code in the Claude desktop app)

This is the Lab 4 part of the chat, tidied up and condensed so it reads clearly. The AI's commands and file edits are summarised in small grey notes instead of full output. The start of the same session, where I gave it exports of my Lab 1 to 3 chats so it could learn my repo layout, is left out.

Me

Yes, the .gitignore will have those two files. Check all the lab folders and file these two the same way. Then start guiding me through Lab 4.

> *Claude checked the lab folders, moved the handout and allocation sheet into lab4-vibecoding/handout/, updated .gitignore and read the assigned repo. Ran: git status --short --ignored. The handout folder shows as ignored, only .gitignore is modified.*

Claude

None of the Lab 1 to 3 folders keep a handout, so Lab 4 gets its own `handout/` folder, and `.gitignore` now skips it:

```
# Lab handouts and class allocation sheets (not my work, allocation has every SRN)
lab*/handout/

# Python
__pycache__/
*.pyc
```

What 17_wordle has. A small terminal Wordle game in `main.py`, `game.py`, `feedback.py` and `words.py`, standard library only. The README lists four tasks:

1. **Duplicate letter feedback.** `feedback.py` marks any letter that appears anywhere in the target as yellow, without counting copies.
2. **Guess handling.** An invalid guess still uses up one of the 6 turns, because the `continue` sits inside `for _ in range(6)`.
3. **4, 5 and 6 letter modes.** `words.py` only has 5 letter words.
4. **History and summary.** History is stored but never printed.

Rules: nothing saved to files, no new dependencies, keep the same modules, keep the full chat history.

Here's how we'll do it. You clone the repo next to your lab repo (not inside it, so there is no nested `.git`), run it once and show me the bug. Then I'll take the before screenshots, fix the four tasks one by one, test them and take the after screenshots. I'll need you to confirm the plan before I change files and check the result at the end.

```
cd "C:\Data\Education\PESU\Semester 5\SE"
git clone https://github.com/SETAPESU26/17_wordle.git
cd 17_wordle
python main.py
```

Me

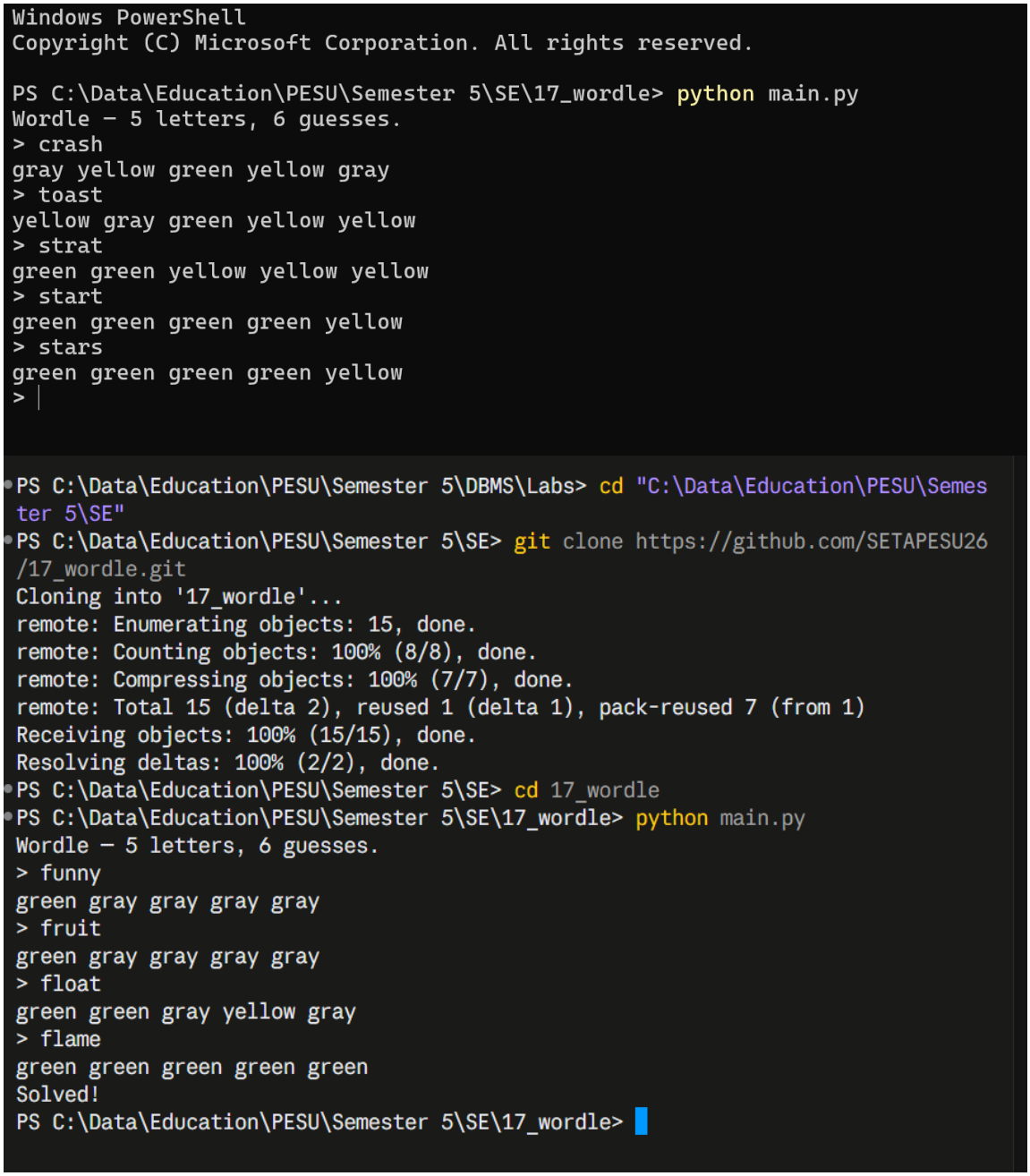
We will do before and after screenshots instead on faculty's instruction. No video recording is essential.

Claude

Screenshots only, no videos. Play a few rounds and send me a screenshot when you see wrong colours.

Me

Here is the bug.



Claude

The first screenshot shows the Task 1 bug three times. The target is `stare`:

Guess	Game shows	Correct Wordle	Why
`toast`	yellow gray green yellow **yellow**	yellow gray green yellow **gray**	`stare` has one `t`, and the first `t` already used it
`strat`	green green yellow yellow **yellow**	green green yellow yellow **gray**	the only `t` was already matched green
`start`	green green green green **yellow**	green green green green **gray**	same reason

Guess	Game shows	Correct Wordle	Why
`stars`	green green green green **yellow**	green green green green **gray**	the only `s` was already matched green

The second screenshot (target `flame`) has no repeated letters, so it doesn't show the bug.

Here's what to do next:

1. **Take the before screenshots in a clean console with the target forced to stare from the command line, so the bug shows every time without changing any code.**
2. **I will fix the four tasks in the clone.**
3. **I will then proceed to add a small test file for the README's required test cases.**
4. **Take after screenshots, one per task.** Screenshots go in `lab4-vibecoding/screenshots/`, numbered so they sort in reading order: `01-before-duplicate-letters.png`, `02-before-invalid-guess.png`, then `03` to `06` for the after shots.

Me

Let's do the Lab 4 work now, task by task. Please:

- fix the code for each task in the scenario README
- make a PDF of this chat's history
- tell me what to push and give me the GitHub link for Lab 4
- add anything else the lab needs

Start.

Claude

This touches more than 3 files, so here's the plan first:

1. **Before screenshots.** Run the original code in a real PowerShell window with the target forced to `stare`, type the guesses in, save 2 PNGs.
2. **Fix the code in the clone.**
   - `feedback.py`: greens first, then count letters for yellows (Task 1)
   - `game.py`: invalid guesses don't use a turn, clean win, loss and quit, pick 4, 5 or 6 letters, history after each guess, summary at the end (Tasks 2 to 4)
   - `words.py`: add 4 and 6 letter words, still in memory
   - `test_wordle.py`: one small file of asserts for the required cases
3. **After screenshots**, one per task.
4. **Copy into your repo** under `lab4-vibecoding/` with `code/`, `screenshots/`, `docs/` and a README, and add a Lab 4 row to the root README.
5. **Chat PDF** of this conversation.
6. **Git commands** and the GitHub link.

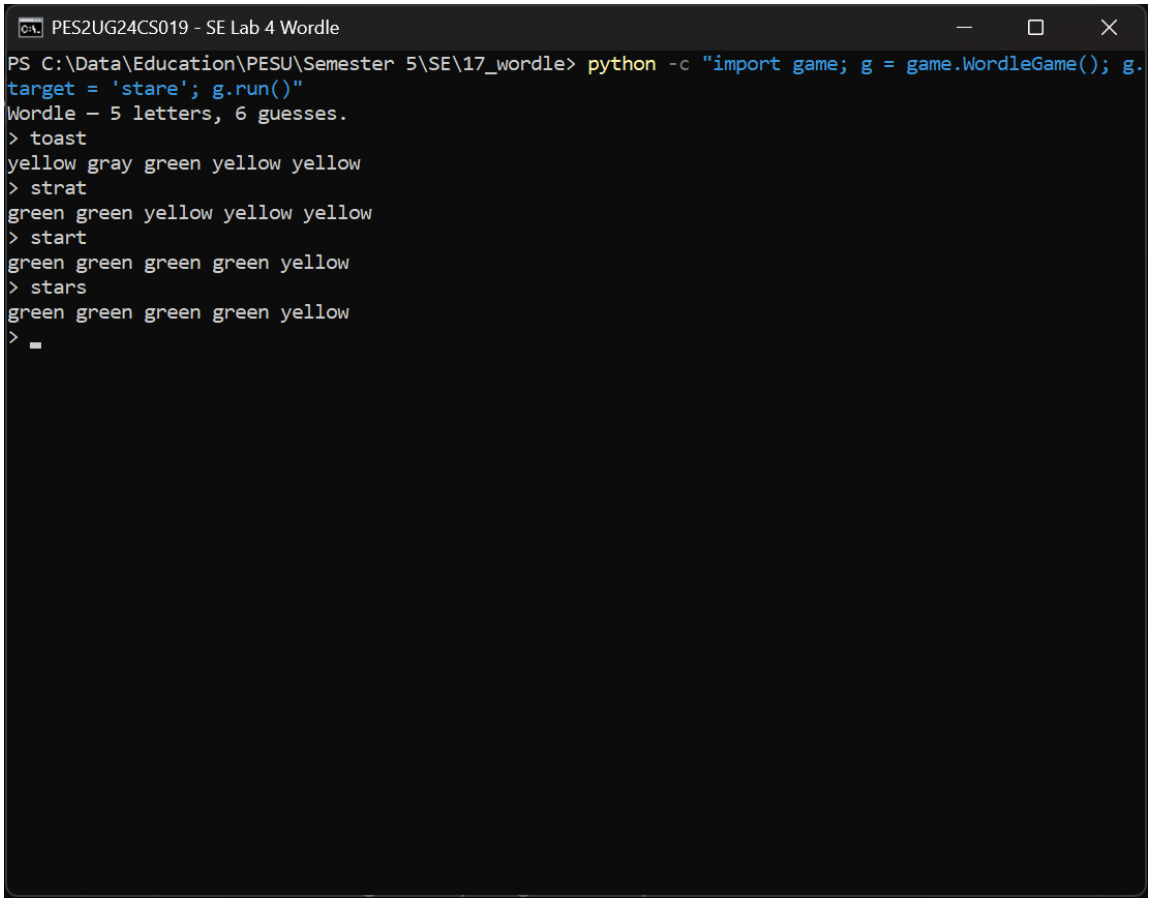
No docx, since the handout only asks for the code, the evidence and the chat history. OK to go ahead?

Me

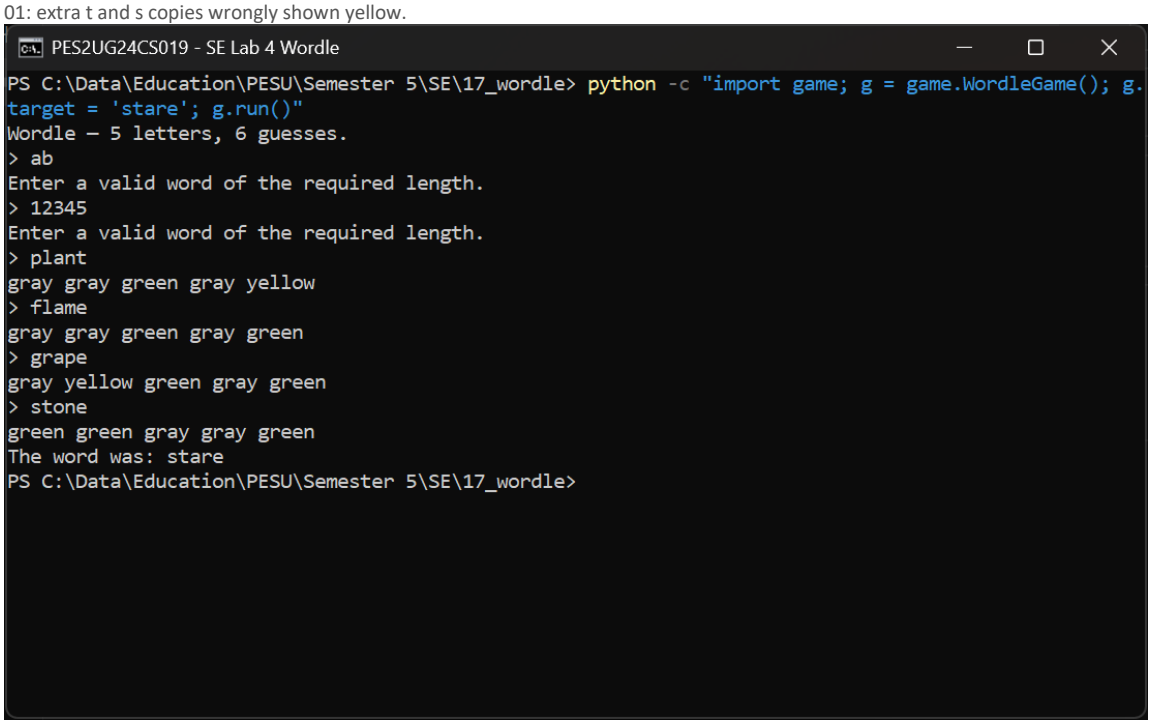
Go ahead with your recommended plan: the code, the screenshots and the chat PDF, no separate docx.

Claude

Before screenshots are done. Both bugs show up clearly.



01: extra t and s copies wrongly shown yellow.



02: two invalid inputs used up turns, so the game ended after only 4 real guesses.

Claude edited feedback.py, words.py, game.py and main.py in the clone.

Claude

The code fixes:

Task 1, `feedback.py`. Exact matches are marked first. Every target letter that wasn't an exact match goes into a `Counter`, and each one can turn at most one guess letter yellow:

```
from collections import Counter

def evaluate(target, guess):
    result = ["gray"] * len(guess)
    unmatched = Counter()
    for i, (t, g) in enumerate(zip(target, guess)):
        if g == t:
            result[i] = "green"
        else:
            unmatched[t] += 1
    for i, g in enumerate(guess):
        if result[i] != "green" and unmatched[g] > 0:
            result[i] = "yellow"
            unmatched[g] -= 1
    return result
```

Task 2, `game.py`. The loop now runs `while len(self.history) < 6`, so only accepted guesses count. Invalid input prints a message and is never added to history. The prompt shows `Guess 3/6`, with clear win, loss and quit messages.

Task 3, `words.py` and `game.py`. Eight 4 letter and eight 6 letter words added to the in-memory list. A length prompt accepts 4, 5 or 6 (Enter means 5). The feedback logic already works for any length.

Task 4, `game.py` and `main.py`. The full history prints after each accepted guess. `play_session()` keeps playing until you stop, then prints a summary: games played, won, lost, quit, average guesses per win, and one line per game.

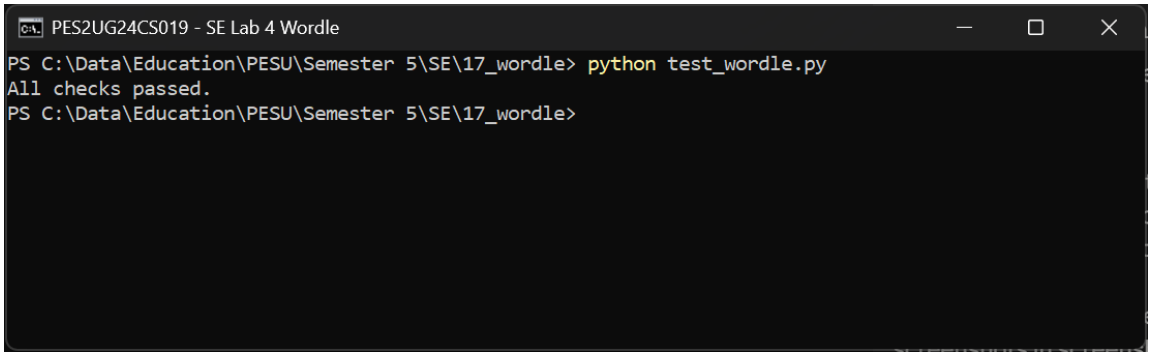
Claude wrote test_wordle.py.

Ran: python test_wordle.py. Output: All checks passed.

Ran: python -c with random.seed(19) to see which words the seeded session picks (apple, then tree).

Claude

`test_wordle.py` covers the README's required testing list: repeated letters both ways round, exact matches, absent letters, invalid lengths, all three word lengths, winning on the sixth guess, losing, quitting and the session summary. Only plain asserts, no test framework.



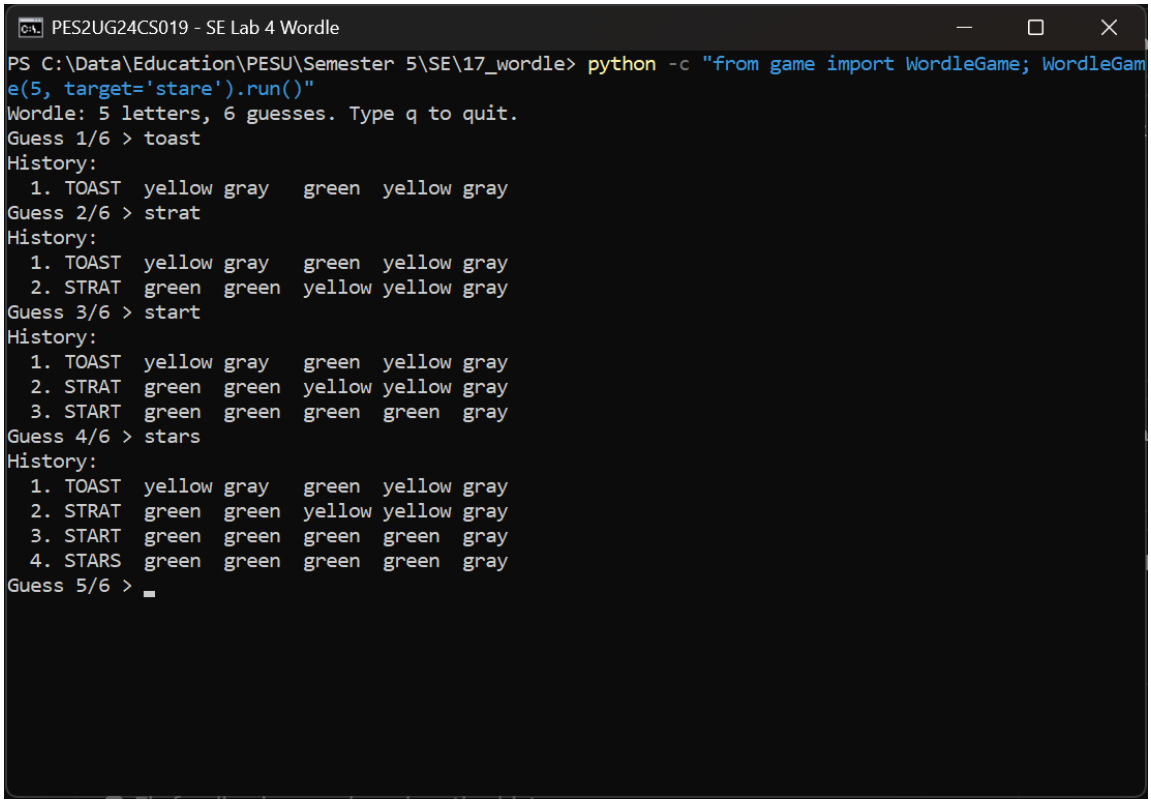
07: all checks pass.

Me

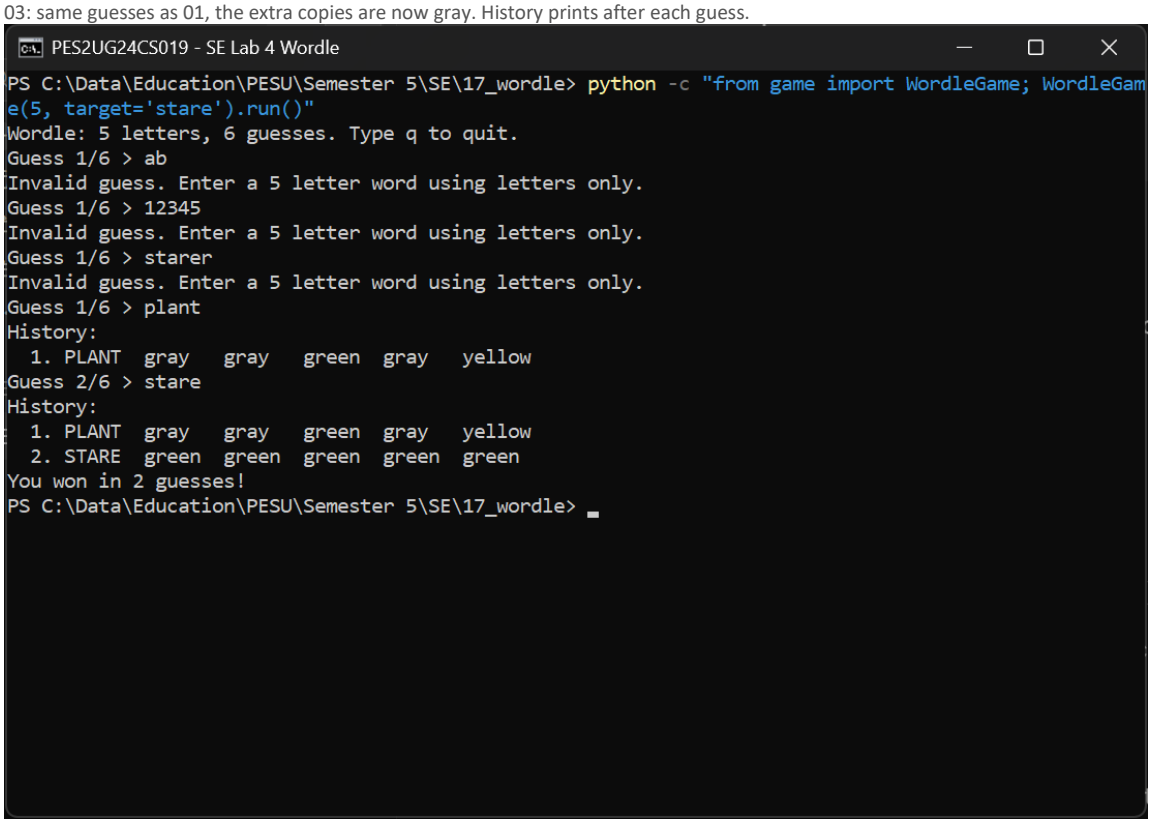
The screenshots are in lab4-vibecoding/screenshots. Please verify them and make the PDF.

Claude

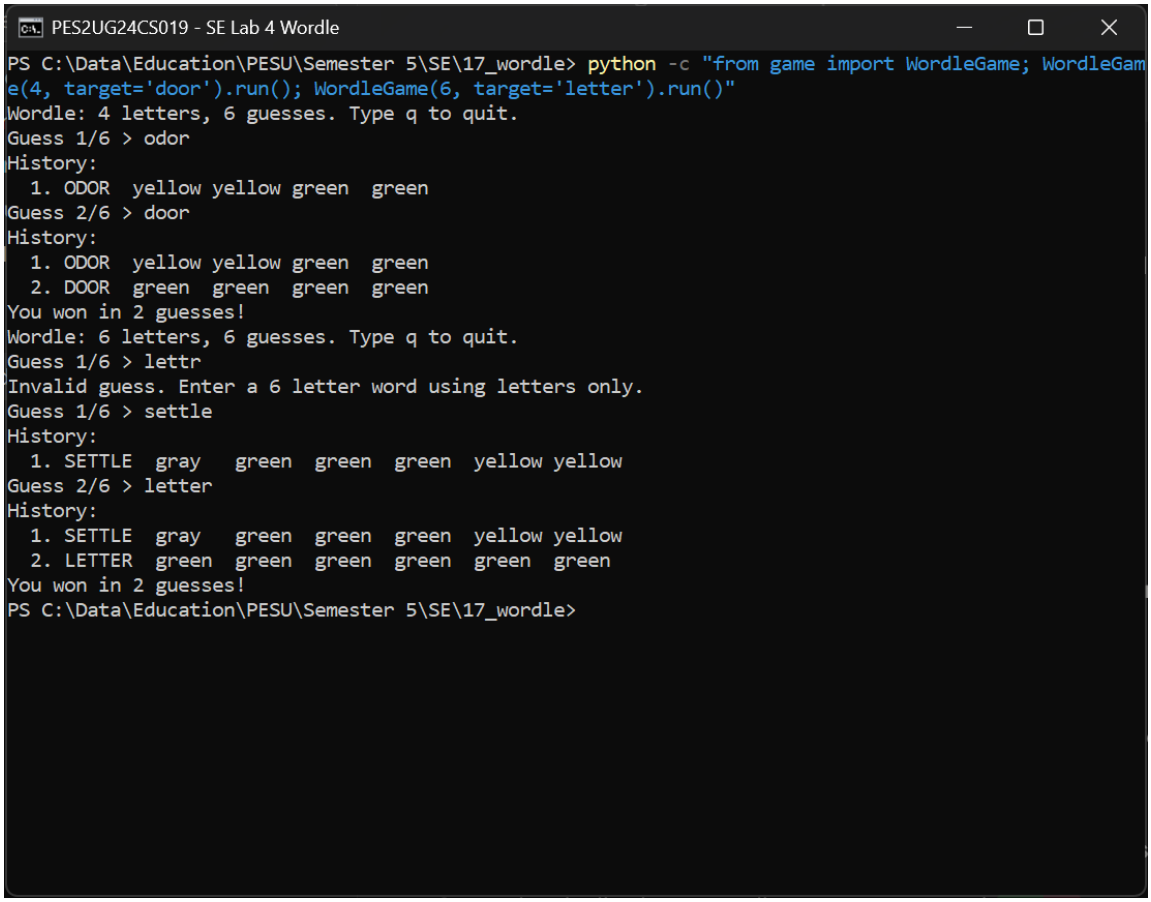
Verified. I checked every colour line against real Wordle rules and they are all correct. The session run in 06 uses random.seed(19) so it can be repeated.



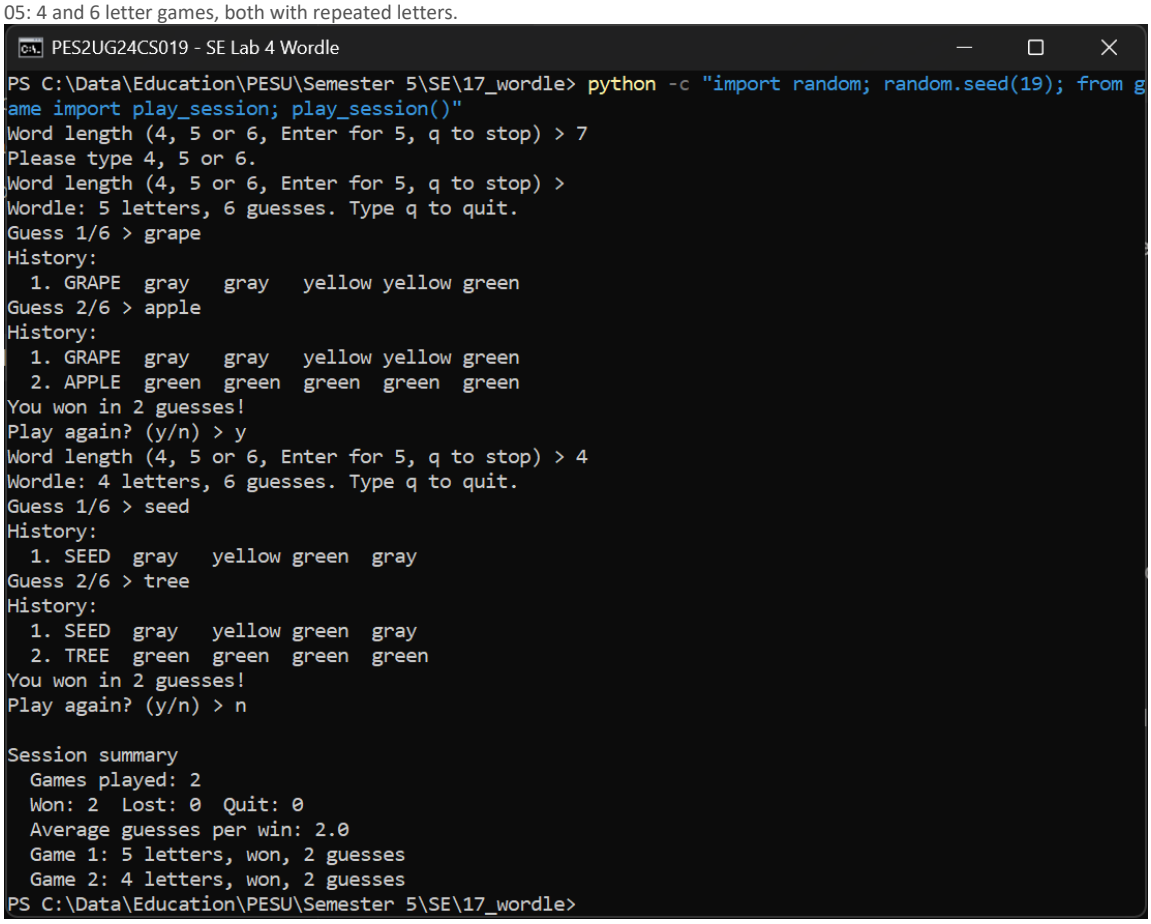
03: same guesses as 01, the extra copies are now gray. History prints after each guess.



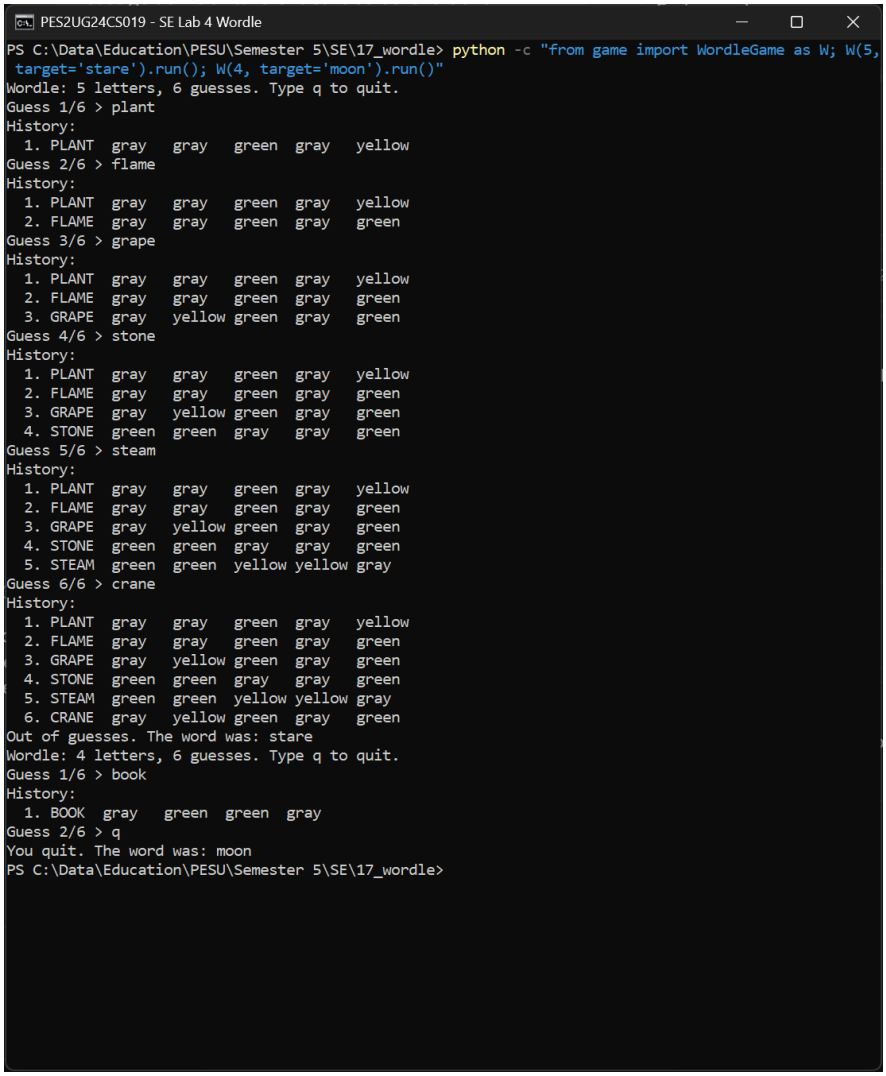
04: invalid inputs stay on Guess 1/6 and don't use a turn.



05: 4 and 6 letter games, both with repeated letters.



06: length prompt, history and the session summary.



08: losing after six accepted guesses, then quitting a 4 letter game.

Claude copied the code (without .git and __pycache__), the screenshots and this PDF into the lab repo, and wrote the Lab 4 README.

Ran: git status --short -uall in the lab repo. Only .gitignore, README.md and the lab4-vibecoding files show up, no handout or __pycache__.

Checked the READMEs and code for em and en dashes. None found.

Claude

Lab 4 is ready to push:

```
lab4-vibecoding/
  code/         fixed game and test_wordle.py
  screenshots/  01-02 before, 03-08 after
  docs/         PES2UG24CS019_SE_Lab4_chat.pdf
  README.md     what was broken, what changed, testing, screenshots
```

The root README has a Lab 4 row and a short paragraph. Push it with:

```
git add .gitignore README.md lab4-vibecoding
git commit -m "Lab 4: vibe coding, fixed Wordle game, screenshots and chat history"
git push origin master
```

Link to submit: https://github.com/UltraBot05/se-ps17-vaccination-scheduling/tree/master/lab4-vibecoding

Don't push from the `17_wordle` clone, since it still points at SETAPESU26. No PR to the main repo.